

Otimização

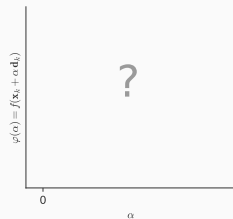
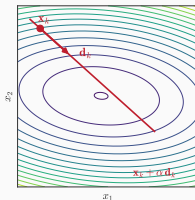
2.1 Otimização unidimensional: definição, classificação e notas gerais

José Firmino Aguilár Madeira

Capítulo 2. Otimização unidimensional — porquê uma só variável?

Os problemas que nos interessam têm n variáveis. Porque estudar otimização em *uma*?

Quase todos os métodos do cap. 3 fazem, em cada iteração, $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$: escolhem uma **direção** $\mathbf{d}_k$ e depois **quanto andar**, α_k .



Capítulo 2. Otimização unidimensional — porquê uma só variável?

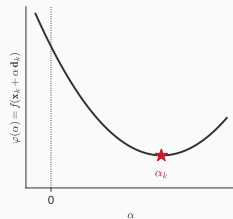
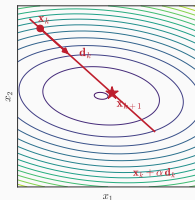
Os problemas que nos interessam têm n variáveis. Porque estudar otimização em *uma*?

Quase todos os métodos do cap. 3 fazem, em cada iteração, $x_{k+1} = x_k + \alpha_k d_k$: escolhem uma **direção** d_k e depois **quanto andar**, α_k .

Com x_k e d_k fixos, $x_k + \alpha d_k$ é uma **reta**, e ao longo dela

$$\varphi(\alpha) = f(x_k + \alpha d_k)$$

é uma função de **uma variável**. Escolher α_k é minimizar φ : a **pesquisa em linha** (*line search*).



Capítulo 2. Otimização unidimensional — porquê uma só variável?

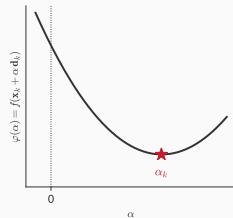
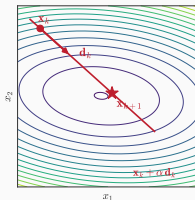
Os problemas que nos interessam têm n variáveis. Porque estudar otimização em *uma*?

Quase todos os métodos do cap. 3 fazem, em cada iteração, $x_{k+1} = x_k + \alpha_k d_k$: escolhem uma **direção** d_k e depois **quanto andar**, α_k .

Com x_k e d_k fixos, $x_k + \alpha d_k$ é uma **reta**, e ao longo dela

$$\varphi(\alpha) = f(x_k + \alpha d_k)$$

é uma função de **uma variável**. Escolher α_k é minimizar φ : a **pesquisa em linha** (*line search*).



Em 1D decidimos **quanto andar**. Em n D temos primeiro de decidir **para onde** andar (cap. 3). E a pesquisa em linha repete-se em cada iteração: no cap. 3 é muitas vezes a maior parte do custo.

O problema: valor mínimo e minimizante

Um problema de otimização **unidimensional**: $\min_{a \leq x \leq b} f(x)$
(a única restrição é o intervalo).

Há duas respostas possíveis:

$$\text{valor mínimo: } f^* = \min_{a \leq x \leq b} f(x)$$

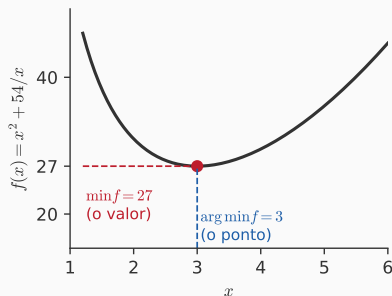
$$\text{minimizante: } x^* = \arg \min_{a \leq x \leq b} f(x)$$

f^* é um valor de f ; x^* é o ponto onde ele é atingido:
 $f(x^*) = f^*$. No exemplo: $\min f = 27$, $\arg \min f = 3$.

Na pesquisa em linha interessa o **ponto**:

$$\alpha_k = \arg \min_{\alpha > 0} \varphi(\alpha).$$

Se houver vários minimizantes, $\arg \min$ designa o conjunto deles; neste capítulo haverá um só.



$f(x) = x^2 + 54/x$: o exemplo que acompanha o capítulo.

O que podemos saber sobre f ?

Que informação temos? Quanto custa obtê-la? O que conseguimos garantir com ela?
(a pergunta que atravessa a UC)

só $f(x)$	ordem 0	→ 2.2 secção áurea: <i>eliminar</i> intervalo; 2.3 Powell: <i>interpol</i> ar
$f(x)$ e $f'(x)$	ordem 1	→ 2.4 bissecção: <i>localizar</i> o zero de f'
f , f' e f''	ordem 2	→ 2.5 Newton: <i>linearizar</i> f'
só f , mas queremos f' , f''		→ 2.6 diferenças finitas: <i>aproximar</i> f' , f''

Mais informação: potencialmente
mais depressa (linear, superlinear,
quadrática)

mas **custa**: derivadas caras, ou pouco fiáveis
se as avaliações têm ruído; e pode tornar
o método **menos robusto** (sem
enquadramento garantido)

O custo de um método mede-se em **avaliações de f** (e de f' , f''), não em operações aritméticas. Contamos à parte n_f (avaliações de f), n_g (de f' ; em $\mathbb{R}^n$, gradientes) e n_H (de f'' ; Hessianas).

Métodos iterativos: quando parar?

Salvo casos triviais, o mínimo não se obtém num só passo. Um método iterativo:

1. parte de uma aproximação inicial x_0 (ou de um intervalo $[a_0, b_0]$);
2. gera uma sucessão $x_1, x_2, \dots$ em que cada termo usa informação de f nos anteriores — só em x_k (Newton), ou em vários (Powell usa três pontos);
3. **para** quando um critério é satisfeito.

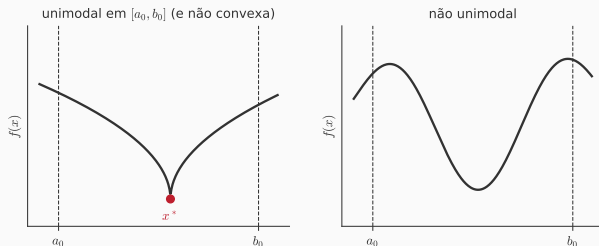
Crítérios de paragem habituais

$$\frac{|x_{k+1} - x_k|}{\max(1, |x_k|)} \leq \varepsilon_x, \quad |f(x_{k+1}) - f(x_k)| \leq \varepsilon_f, \quad |f'(x_k)| \leq \varepsilon_g, \quad b_k - a_k \leq \varepsilon_x, \quad k \geq k_{\max}.$$

Usar sempre $k_{\max}$ (protege contra métodos que não convergem). Dividir por $\max(1, |x_k|)$ torna o critério relativo quando $|x_k|$ é grande e absoluto perto de zero; nos métodos de intervalo usa-se o comprimento $b_k - a_k$.

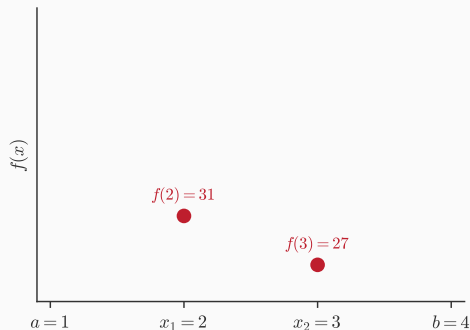
Definição

f é **unimodal** em $[a_0, b_0]$ se tem um e um só mínimo local x^* nesse intervalo: é estritamente decrescente em $[a_0, x^*]$ e estritamente crescente em $[x^*, b_0]$.



Não exige continuidade, derivadas nem convexidade — basta a forma “desce, depois sobe”. Toda a função estritamente convexa é unimodal; a recíproca é falsa (à esquerda: unimodal, não convexa e sem derivada em x^*).

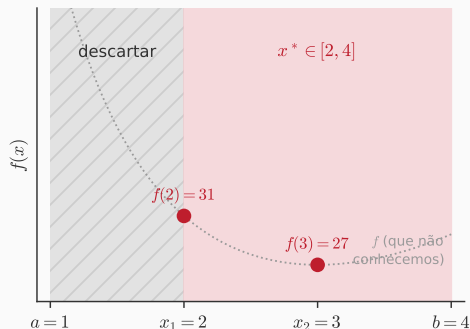
Só dois valores de f : o que podemos deitar fora?



f é unimodal em $[1, 4]$, mas não a conhecemos: só temos $f(2) = 31$ e $f(3) = 27$.

De que parte do intervalo temos a certeza de que não precisamos?

Só dois valores de f : o que podemos deitar fora?

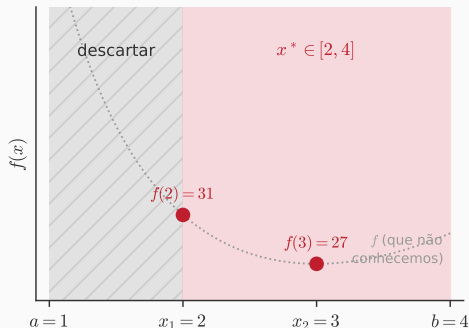


f é unimodal em $[1, 4]$, mas **não a conhecemos**: só temos $f(2) = 31$ e $f(3) = 27$.

De que parte do intervalo temos a certeza de que não precisamos?

Como $f(2) > f(3)$, o mínimo não está em $[1, 2]$: se estivesse, f subiria de 2 para 3. Logo $x^* \in [2, 4]$ — um terço do intervalo descartado com duas avaliações e *sem derivadas*.

Só dois valores de f : o que podemos deitar fora?



f é unimodal em $[1, 4]$, mas **não a conhecemos**: só temos $f(2) = 31$ e $f(3) = 27$.

De que parte do intervalo temos a certeza de que **não precisamos?**

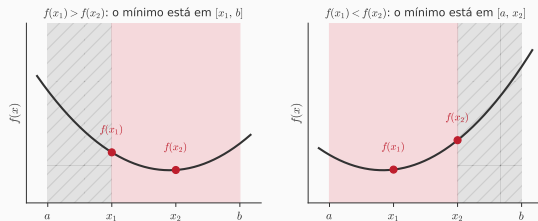
Como $f(2) > f(3)$, o mínimo não está em $[1, 2]$: se estivesse, f subiria de 2 para 3. Logo $x^* \in [2, 4]$ — um terço do intervalo descartado com duas avaliações e *sem derivadas*.

E $x_2 = 3$ fica **dentro** do novo intervalo, com f já calculada.

Se vamos repetir isto muitas vezes, onde colocar x_1, x_2 para **reutilizar** uma avaliação? → 2.2

A regra geral: eliminar intervalo

Com f unimodal em $[a, b]$ e dois pontos interiores $x_1 < x_2$:

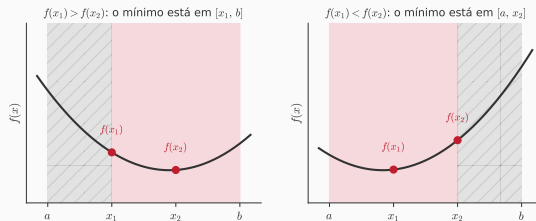


$$f(x_1) > f(x_2) \Rightarrow x^* \in [x_1, b], \quad f(x_1) < f(x_2) \Rightarrow x^* \in [a, x_2], \quad f(x_1) = f(x_2) \Rightarrow x^* \in [x_1, x_2].$$

Robusto (enquadramento garantido): o intervalo contém sempre x^* , e encolhe de forma previsível.
Só se usa a **ordem** dos dois valores ($<$ ou $>$).

A regra geral: eliminar intervalo

Com f unimodal em $[a, b]$ e dois pontos interiores $x_1 < x_2$:



$$f(x_1) > f(x_2) \Rightarrow x^* \in [x_1, b], \quad f(x_1) < f(x_2) \Rightarrow x^* \in [a, x_2], \quad f(x_1) = f(x_2) \Rightarrow x^* \in [x_1, x_2].$$

Robusto (enquadramento garantido): o intervalo contém sempre x^* , e encolhe de forma previsível. Só se usa a **ordem** dos dois valores ($<$ ou $>$).

E se usássemos também a sua *magnitude*? Com três valores de f pode ajustar-se uma parábola e estimar onde está o mínimo $\rightarrow$ 2.3 (Powell).

Com derivadas: procurar o zero de f'

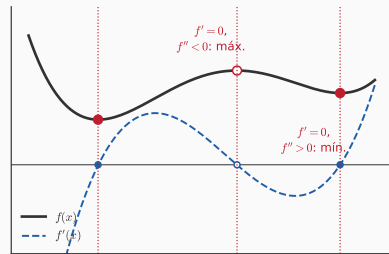
Se f é diferenciável, num mínimo interior $f'(x^*) = 0$ (condição necessária); se além disso $f''(x^*) > 0$, x^* é mínimo local estrito (condição suficiente).

Em vez de minimizar f , podemos procurar o zero de f' :

- **bisseção** sobre f' (2.4): o *signal* de f' diz de que lado está o zero. Mantém um intervalo com $f'(a) < 0 < f'(b)$ (f' contínua);
- **Newton** sobre f' (2.5): a inclinação de f' , isto é f'' , permite *estimar onde* está o zero:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Atenção: $f' = 0$ também em máximos e inflexões — verificar f'' ($f'' = 0$ é inconclusivo).

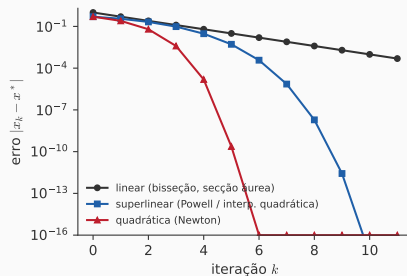


Velocidade de convergência

Seja $e_k = |x_k - x^*|$. Diz-se que a convergência é

- **linear** se $e_{k+1} \leq c e_k$, $0 < c < 1$ — ganha um número fixo de dígitos por iteração. Nos métodos de intervalo é o *comprimento* L_k que se reduz linearmente ($L_{k+1} = 0,5L_k$ na bissecção, $0,618L_k$ na secção áurea) e $e_k \leq L_k/2$;
- **superlinear** se $e_{k+1}/e_k \rightarrow 0$ (interpolação quadrática: ordem $\approx 1,3$);
- **quadrática** se $e_{k+1} \leq c e_k^2$ — perto de x^* , o número de dígitos corretos *tende a duplicar* por iteração (Newton).

O preço da velocidade: Newton precisa de f'' e de x_0 próximo; com as respectivas hipóteses, bissecção e secção áurea têm convergência garantida.



Mapa do capítulo: cinco ideias

Método	Informação	Ideia	Garantia / velocidade
2.2 Secção áurea	f	eliminar intervalo	robusta (f unimodal); linear, 0,618 por avaliação
2.3 Powell	f	interpoliar uma parábola	rápida perto do mínimo (superlinear); sozinha, sem garantia
2.4 Bisseção	f'	localizar o zero de f'	robusta (zero único enquadrado); linear, 0,5
2.5 Newton	f', f''	linearizar f'	quadrática local; pode divergir
2.6 Diferenças finitas	f	aproximar f', f''	erro de truncatura + erro de arredondamento

Na prática combinam-se: um método robusto (enquadramento garantido) localiza o mínimo, um rápido refina-o.

secção áurea + interpolação parabólica → fminbnd
→ pesquisa em linha do cap. 3 (gradiente, Newton amortecido, GC)

O cap. 2 não é só “matéria anterior”: explica uma peça que trabalha *dentro* dos algoritmos do cap. 3.

O QUE DEVE FICAR — 2.1

1. Em $\mathbb{R}^n$, nos métodos baseados em direções, escolhe-se uma direção e depois quanto andar: $\alpha_k = \arg \min_{\alpha > 0} \varphi(\alpha)$, com $\varphi(\alpha) = f(x_k + \alpha d_k)$ — um problema de **uma** variável.
2. $\min f$ é o *valor* mínimo; $\arg \min f$ é o *ponto* onde é atingido.
3. Os métodos distinguem-se pela **informação** que usam (só f , f' , f''), pelo que ela custa e pelo que garante: mais informação pode dar convergência mais rápida (linear $\rightarrow$ superlinear $\rightarrow$ quadrática), mas custa e pode tornar o método menos robusto (sem enquadramento garantido).
4. Com f unimodal, **dois valores** de f chegam para descartar parte do intervalo (só se usa a sua ordem); com derivadas, minimizar f passa a **procurar o zero de f'** ($f' = 0$ também em máximos: verificar f'').

A seguir (2.2): onde colocar x_1 e x_2 para que cada redução do intervalo custe **uma só** avaliação de f ?

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Deb, K., *Optimization for Engineering Design: Algorithms and Examples*, 2.^a ed., PHI Learning, 2012 — cap. 2.
- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2.^a ed., Springer, 2006 — cap. 3, pesquisa em linha.

Recursos

- MATLAB: `fminbnd`.